Experiment Response Monitor — Project Proposal

Oct 8, 2026 · @bhavika

Proposing "Experiment Response Monitor" as my Level 3 (advanced) project for the no-code/n8n system design course: a workflow that watches live experiment responses as they arrive and flags the ones that need a person to look at them, instead of relying on someone watching the dashboard. Sharing this for sign-off before I start building.

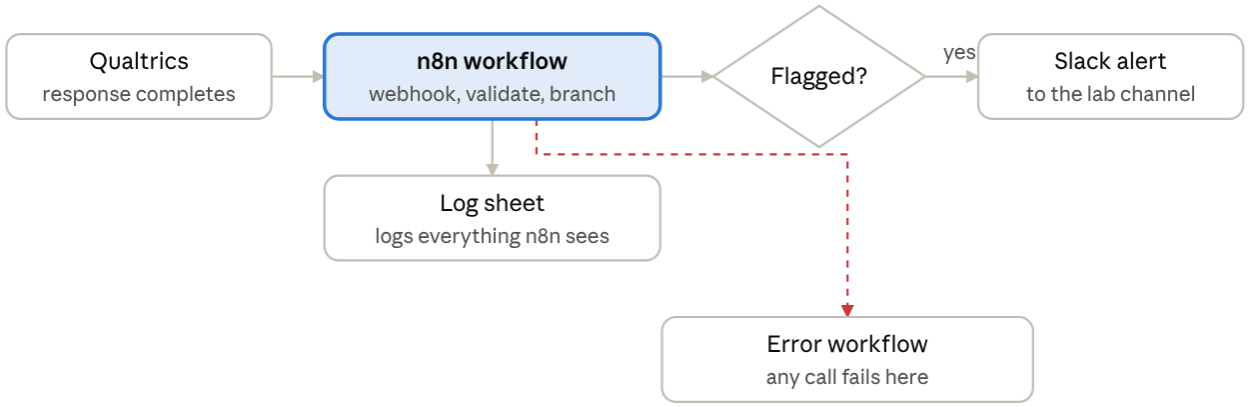
The problem

During a live session (the kind of Qualtrics study the lab runs, e.g. VRAC), responses arrive one at a time while the session is still open. Catching a bad one today means a research assistant watching the dashboard by eye — a skipped attention check, a duplicate submission, a response filed under the wrong condition. Nothing alerts anyone in real time; problems usually surface during cleanup, days after the session closes.

Four questions set the scope:

- **What happens in the real world:** a participant submits a response in Qualtrics; it should be checked against a few rules the moment it lands, and the team should hear about anything off immediately, not at cleanup.
- **What data exists, and where:** the response — participant ID, session ID, timestamp, embedded condition data, attention-check answer, completion duration — lives in Qualtrics. This project reads that data; it doesn't own it.
- **What triggers what:** a completed response in Qualtrics is the only trigger. No clock, no human click.
- **What's allowed to break:** the monitor, never the survey. If the monitor goes down, participants keep completing the study with zero disruption — it just means the team catches problems later instead of immediately, which is where things stand today anyway.

Qualtrics feeds n8n, which logs every response and alerts Slack only on a flag



Experiment Response Monitor — architecture, 6 blocks

Six blocks, same ones from the course framework: Qualtrics is the external service and the database of record; n8n is the backend; the log sheet is a cache of what n8n has already seen, never the source of truth; Slack is the alerting channel, reached only on a flag. The dashed red path is the one failure case that gets its own alert — the monitor breaking, not the data.

Validation rules

Every response runs through the same switch, in order. The first rule it matches decides the flag; a response matching none of them is clean.

Rule	Condition	Flag
Incomplete	A required field is missing	`incomplete`
Attention check	The attention-check item was answered wrong	`attention_check_fail`
Duration outlier	Completion time is below/above the session's expected range	`duration_outlier`
Duplicate	Response or participant ID already in the log	`duplicate`
Branch mismatch	Recorded condition doesn't match the embedded assignment	`branch_mismatch`
None of the above	—	`clean`, logged, no alert

This is the Level 3 part of the scope: five branches on one Switch node, not a single If, each one a real rule from running these sessions rather than a toy example.

Failure handling

Failure	Setting	Behaviour
Qualtrics or Slack call times out (5xx/429)	Retry On Fail	Retries automatically; never retries a 4xx
A node in the chain dies	On Error: continue using error output	Routes to the error workflow instead of killing the run
Nobody notices it's broken	Error Trigger → error workflow	A separate Slack alert, so a dead monitor is itself visible
Slack alert itself fails to send	Retry 3x, then log	Written to an `alert_failed` column in the sheet — never silently dropped

Two rules this is built around: the monitor must never block or slow down the survey — it only reads Qualtrics, it never writes back to it — and a flag that fails to send has to leave a trace somewhere a human will see it, even if that trace is just a row in a sheet nobody got pinged about.

Scope for this version

- **Won't touch the live survey.** The monitor only reads from Qualtrics; it never modifies, pauses, or gates a participant's session. Worst case it goes down and nobody gets an alert — the study itself keeps running exactly as it would without it.
- **Qualtrics stays the system of record.** The log sheet is a cache of what's already been flagged, built to catch duplicate alerts — not a second copy of the actual response data.
- **No AI in v1.** Every rule above has a deterministic right answer, so each is an If/Switch check, not a model call. A v2 idea — an AI-drafted note explaining *why* a response was flagged, with a human still deciding — is worth a follow-up proposal once the rule-based version is running.
- **One study at a time.** Built against the current Qualtrics survey structure; adapting it to a second study is follow-on work, not part of this submission.

Timeline

Week	Deliverable
1	Qualtrics webhook wired up in a dev n8n instance; all five validation rules built and tested against mock responses
2	Connected to a real (non-live) Qualtrics session; thresholds tuned; Slack alerting built and tested
3	Idempotency check, error workflow, and the production/dev split added; documentation written; demo for Session 3

What I need from you

Sign-off to start building against this design, or anything you'd want changed first — scope, the rule set, or whether this reads as Level 3 to you. Happy to adjust before week 1 starts.